Task 1 - Web App QA & Debug Report

Mukul Choudhary | 30 September 2026 | Local API assessment

Application: gothinkster/node-express-realworld-example-app, commit 30b68e1e881462b2f4164ea09ab4c4f5699c7b0b. Local base URL: http://localhost:3000/api.

Environment and scope: Node 24.19.0, Prisma 4.16.2, SQLite. Only the datasource provider and local launch dependencies were adapted; business logic was unchanged. The supplied live demo returned 404/NoSuchBucket in this browser, so a local implementation was used. Findings below are reproduced through HTTP requests, with source inspection supporting suspected causes. Full upstream frontend accessibility/performance coverage is outside this report.

Flows checked: signup 201, login 200, create 201, edit 200, delete 204, deleted article lookup 404, and request without session token 401. Logout was checked as client-side token removal; server-side token revocation was not claimed. Evidence: evidence/qa-results.json and scripts/qa_check.py.

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
1	Body-only edit removes all article tags	Sign up and log in. POST /api/articles with tagList ["python","qa"]. PUT /api/articles/{slug} with {"article":{"body":"Edited body"}}.	Expected: body changes; existing tags remain. Actual: HTTP 200, body updated, tagList becomes []. This can break categorization and discovery.	High	updateArticle unconditionally calls disconnectArticlesTags; an omitted tagList becomes [].
2	Whitespace-only article content accepted	With a valid token, POST /api/articles using title, description and body equal to three spaces.	Expected: reject empty content with field-specific 422. Actual: HTTP 201, whitespace content stored; resulting slug is "-1" in the recorded run.	Medium	createArticle uses truthiness checks instead of validating trimmed strings.
3	Invalid email accepted during registration	POST /api/users with unique username, password "x" and email "invalid-e4653bcf" (no @ or domain); use a new suffix on rerun.	Expected: invalid email rejected with a useful validation error. Actual: HTTP 201 and malformed email stored; account contact details are unusable.	Medium	createUser checks only whether email is non-empty; no format validation.
4	One-character password accepted	Register a new test account with a valid unique email and password "x". An independent valid-email probe is included in the evidence.	Expected: apply a defined minimum-length policy and return 422. Actual: independent valid-email registration returns HTTP 201 with password "x". No minimum is enforced in source.	Medium	createUser only checks non-empty trimmed password before hashing. Minimum policy is a production assumption, not specified by RealWorld.

Bug register - continued

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
5	Missing user payload returns internal error	POST /api/users/login with JSON {} and Content-Type application/json.	Expected: 400/422 with missing user/email/password fields. Actual: HTTP 500 and "Cannot read properties of undefined (reading email)". Users receive an internal error instead of guidance.	Medium	Controller passes req.body.user directly; login accesses userPayload.email without validating the object.
6	Saved biography cannot be cleared	PUT /api/user with bio "Temporary QA biography". Then PUT with bio "". GET /api/user.	Expected: empty bio persists. Actual: HTTP 200 on both updates, but the previous biography remains after clear and subsequent GET.	Medium	updateUser spreads bio only when truthy; it cannot distinguish omitted from intentionally empty.
7	Malformed JSON reported as server failure	POST /api/users with raw body {bad-json and Content-Type application/json.	Expected: HTTP 400 and a safe parse-error message. Actual: HTTP 500 with JSON parser error text. This misclassifies client errors and complicates monitoring.	Low	Global error middleware ignores body-parser status/statusCode and defaults unknown errors to HTTP 500.

Root-cause analysis - article tags disappear on edit

A body-only article update succeeds but removes the existing tags. The recorded request omitted tagList, and the response returned an empty list after originally storing python and qa. In updateArticle, an absent or empty tagList is converted to an empty relation list. The service then calls disconnectArticlesTags before updating the article, regardless of whether tags were supplied. This treats an omitted field as an instruction to delete data and can break category filtering. I would distinguish an omitted tagList from an explicitly supplied array, validate the array, and preserve existing tags when it is absent. When tags are supplied, I would replace the relation inside the same atomic update or transaction as the content change, avoiding a partially cleared relation if another write fails. I would verify body-only edits preserve tags, an explicit empty list clears tags, a replacement list is saved, and a failed update leaves the original article unchanged.

Priorities for a non-technical stakeholder: Fix tag loss first to prevent accidental content changes. Next, make account/content validation and profile editing predictable, then return appropriate client-error messages. These are proposed fixes; the tested upstream application has not been patched.

Source: https://github.com/gothinkster/node-express-realworld-example-app
Assumptions: valid contact email and a minimum password length are production expectations. JSONPlaceholder workflow sample data is unrelated to QA findings. Original evidence uses only disposable local accounts and contains no authentication tokens.